Luiss
Libera Università Internazionale
degli Studi Sociali Guido Carli

Lab Cryptography Toolkit

Data Privacy and Security

Data Science and Management (DASMA)

LUISS

Crypto CTF Toolkit - Encoding

Operation	Arguments	What it does	Library
`.encode()`	none (called on string)	Converts a string into bytes	Built-in
`.decode()`	none (called on bytes)	Converts bytes into a string	Built-in
`bytes(...)`	arg1: int/string, arg2: encoding (optional)	Creates an immutable bytes object	Built-in
`bytearray(...)`	arg1: int/string, arg2: encoding (optional)	Creates a mutable byte sequence	Built-in
`int.from_bytes()`	arg1: bytes, arg2: byteorder	Converts bytes to an integer	Built-in
`int.to_bytes()`	arg1: length, arg2: byteorder	Converts an integer to bytes	Built-in
`bytes_to_long()`	arg1: bytes	Converts bytes to a large integer	PyCryptodome
`long_to_bytes()`	arg1: integer	Converts a large integer to bytes	PyCryptodome
`bytes.fromhex()`	arg1: hex string	Converts hex strings into bytes	Built-in
`binascii.unhexlify()`	arg1: hex string	Converts hex strings into bytes	binascii
`binascii.hexlify()`	arg1: bytes	Converts bytes into hex	binascii
`hex()`	none (called on int/bytes)	Converts bytes/int to a hex string	Built-in
`base64.b64decode()`	arg1: bytes/str	Converts Base64 to data	base64
`base64.b64encode()`	arg1: bytes	Encodes data in Base64	base64

Crypto CTF Toolkit - Operations

Operation	Arguments	What it does	Library
`zip()`	arg1: iterable1, arg2: iterable2, ...	Iterates over byte sequences in parallel	Built-in
`^`	arg1: int, arg2: int	XOR operator – performs bitwise XOR on integers	Built-in
`ord()`	arg1: character	Converts a character to an integer	Built-in
`chr()`	arg1: integer	Converts an integer to a character	Built-in
`strxor()`	arg1: bytes, arg2: bytes	XORs two byte sequences (same len)	PyCryptodome
`xor()`	arg1: bytes, arg2: bytes	XORs two byte sequences (any len)	Pwntools

Crypto CTF Toolkit - Math Helpers

Operation	Arguments	What it does	Library
`%`	arg1: int, arg2: int	Computes the remainder modulo n	Built-in
`//`	arg1: int, arg2: int	Integer division (discard remainder)	Built-in
`**`	arg1: int, arg2: int	Exponentiation	Built-in
`pow()`	arg1: base, arg2: exponent, arg3: modulus (optional)	Modular exponentiation	Built-in
`math.gcd()`	arg1: int, arg2: int	Computes greatest common divisor	math
`inverse()`	arg1: int, arg2: modulus	Computes modular inverse	PyCryptodome
`getPrime()`	arg1: bit length	Generates prime of specified bit len	PyCryptodome
`isPrime()`	arg1: int	Checks whether a number is prime	PyCryptodome

Crypto Challenges: Rule of Thumb

> Crypto operates on bytes. Humans read strings. Math prefers integers.

Strings (Human-readable)

Examples:

- **ASCII** strings - "FLAG{CRYPT0_1S_FUN}"
- **hex** strings - 4d79537472696e67
- **base64** strings - U0VMRUNUIFBBQ0s=

> **NOTE**: Must be **converted to bytes** before crypto operations!

Crypto Challenges: Rule of Thumb

> Crypto operates on bytes. Humans read strings. Math prefers integers.

Bytes (Crypto-native)

Examples: b'\x46\x4c\x41\x47', b'Hello World'

Used for:

- Bitwise operations e.g. XOR, AND, NOT, ...
- Encryption / decryption.
- Hashing.

Bytes

> At the lowest level, all digital data is stored as bytes, where each byte is a group of 8 bits (0s and 1s).

Bits (0, 1)

↓

Bytes (8 bits = 256 possible values)

↓

Data types (integers, floats, characters, booleans)

↓

...

Bytes can be **encoded** to be represented in different formats (e.g. ASCII, base64, hex, ...)

- Encoding ≠ Encryption.
- Encoding simply means you are changing how the same data is represented, **not hiding it**.

Bytes ↔ Strings

> Python provides a dedicated `bytes` type to work with raw binary data.

You can move between strings and bytes using `encode()` and `decode()`:

String → Bytes (**encode**)

```
s = "Hello World"
b = s.encode()      # returns byte object b'Hello World'
```

Bytes → String (**decode**)

```
s2 = b.decode()     # returns "Hello World"
```

Bytes Literal

Alteratively, you can create a `bytes` object directly using a **bytes literal**:

```
b = b'Hello World'
```

- The **b prefix** tells Python this is a bytes object, not a string.
- Each character in the literal corresponds to a byte value.
- The produced byte object is **immutable**.

> **NOTE**: byte literals only support ASCII characters:

This works:

```
b'Hello'
```

These DON'T:

```
b'03f4'      # hex-encoded string
b'SGVsbG8='  # base64-encoded string
b'Café'      # Contains NON-ASCII char
```

Bytes ↔ Encoded Strings

To convert encoded strings into bytes, you must decode them using the appropriate method:

Hex → Bytes (equivalent alternatives):

```
bytes.fromhex('03f4')
```

```
from binascii import unhexlify
unhexlify('03f4')
```

Base64 → Bytes

```
import base64
base64.b64decode('SGVsbG8=')
```

NOTE: The decoding method must match the encoding used.

Bytes Sequences

> What if you need to modify individual bytes in Python?

`bytes([...])` -> produces an **immutable** byte object.

```
bytes("Hello", "ascii")  #string + encoding
bytes([65, 66, 67])      #array of integers in the range 0-255
```

`bytearray(...)` -> produce a **mutable** byte object.

```
b = bytearray(b'ABC')
b[0] = 97
```

- Similar to bytes, but **modifiable**.
- Useful when you need to change individual byte values.

Bytes Order (Endianess)

> Endianness defines how bytes are stored in memory.

- **Big-endian**: most significant byte first ≈ "human reading order".
- **Little-endian**: least significant byte first ≈ "inverse order".

Example: consider the word

```
b'HELLO'  # [72, 69, 76, 76, 79] = ASCII integers for each letter
```

Big-endian -> bytes stored in text order

```
H  E  L  L  O
72 69 76 76 79
```

Little-endian -> bytes stored in reverse order

```
O  L  L  E  H
79 76 76 69 72
```

Bytes ↔ Integers

Python lets you convert between integers and bytes easily:

Bytes → Integer

```
b = b'HELLO'     # [72, 69, 76, 76, 79]
n = int.from_bytes(b, byteorder='big')  #or byteorder='little' for litte-endian
```

- Interprets the bytes as one single number
- **Must specify byteorder**: big-endian and little-endian produce different integers.

Integer → Bytes

```
n = 310400273487
b = n.to_bytes(5, byteorder='big')  #5 = n. of bytes to use to represent the integer
```

Working with Encodings - ASCII

Standard encoding that maps each character to a unique number between 0 and 127.

40	(	50	2	60	<	70	F	80	P	90	Z	100	d	110	n
41	)	51	3	61	=	71	G	81	Q	91	[	101	e	111	o
42	*	52	4	62	>	72	H	82	R	92	\	102	f	112	p
43	+	53	5	63	?	73	I	83	S	93	]	103	g	113	q
44	,	54	6	64	@	74	J	84	T	94	^	104	h	114	r
45	-	55	7	65	A	75	K	85	U	95	_	105	i	115	s
46	.	56	8	66	B	76	L	86	V	96	`	106	j	116	t
47	/	57	9	67	C	77	M	87	W	97	a	107	k	117	u
48	0	58	:	68	D	78	N	88	X	98	b	108	l	118	v
49	1	59	;	69	E	79	O	89	Y	99	c	109	m	119	w

Uppercase and lowercase letters have different ASCII values.

- Example: 'A' → 65, 'a' → 97.

Working with Encodings - ASCII

You can move between characters and their ASCII values using these functions:

- `ord(character)` - returns the number (ASCII value) of a character:

```
ord('A')  # returns 65
```

- `chr(number)` - returns the character corresponding to the passed number (ASCII value):

```
chr(65)   # returns 'A'
```

Working with Encodings – Hex

Hexadecimal (hex) is a way to represent bytes using **base-16 digits** (0–9, a–f).

- Used to display binary data (bytes) in a readable form.
- Each byte is represented by **two hex characters**.

Example:

```
b = b'Hi'

#ASCII values:
# H -> 72
# i -> 105

b.hex() = 48 69
#48 = 2 hex characters = the single H byte.
#69 = 2 hex characters = the single i byte.
```

INTUITION: strings containing only the characters (0-9, a-f) are likely hex-encoded.

DECIMAL	HEX	BINARY
0	0	0000
1	1	0001
2	2	0010
3	3	0011
4	4	0100
5	5	0101
6	6	0110
7	7	0111
8	8	1000
9	9	1001
10	A	1010
11	B	1011
12	C	1100
13	D	1101
14	E	1110
15	F	1111

Working with Encodings – Hex

You can move between bytes and hex strings using these functions:

Bytes → Hex

```
b = b'Hi'
b.hex()  # returns '4869'
```

```
import binascii
binascii.hexlify(b'Hi')
# returns b'4869'
```

Hex → Bytes

```
bytes.fromhex('4869')
# returns b'Hi'
```

```
import binascii
binascii.unhexlify(b'4869')
# returns b'Hi'
```

Working with Encodings – Base64

Base64 is a way to represent bytes using **64 ASCII characters** (A–Z, a–z, 0–9, +, /) plus = for padding.

- Used to encode binary data (bytes) in a form that can be safely transmitted.
- Each 3 bytes of data become **4 Base64 characters**.
- Padding (=) is added when the number of bytes isn't a multiple of 3.

Example:

```
b = b'Hi'

import base64
base64.b64encode(b)    # b'SGk='
```

INTUITION: strings containing only letters, digits, +, / (and possibly = at the end) are likely base64-encoded.

Base64 Encoding Table

Value	Char	Value	Char	Value	Char	Value	Char
0	A	16	Q	32	g	48	w
1	B	17	R	33	h	49	x
2	C	18	S	34	i	50	y
3	D	19	T	35	j	51	z
4	E	20	U	36	k	52	0
5	F	21	V	37	l	53	1
6	G	22	W	38	m	54	2
7	H	23	X	39	n	55	3
8	I	24	Y	40	o	56	4
9	J	25	Z	41	p	57	5
10	K	26	a	42	q	58	6
11	L	27	b	43	r	59	7
12	M	28	c	44	s	60	8
13	N	29	d	45	t	61	9
14	O	30	e	46	u	62	+
15	P	31	f	47	v	63	/

Working with Encodings – Base64

You can move between bytes and base64 strings using these functions:

Bytes → Base64

```
import base64
base64.b64encode(b'Hello World')  # b'SGVsbG8gV29ybGQ='
```

Base64 → Bytes

```
import base64
base64.b64decode(b'SGVsbG8gV29ybGQ=')   # b'Hello World'
```

XOR Operator

The ^ operator in Python performs a bitwise XOR **between two integers**.

- Works one byte at a time when applied to byte objects.
- **Cannot be applied directly to byte sequences**:

This does NOT work:

```
b1 = b'AB'
# bytes object: [65, 66]
b2 = b'cd'
# bytes object: [99, 100]
b1 ^ b2  #TypeError
```

This works:

```
b1 = b'B'   # 66
b2 = b'c'   # 99
result = b1[0] ^ b2[0]
```

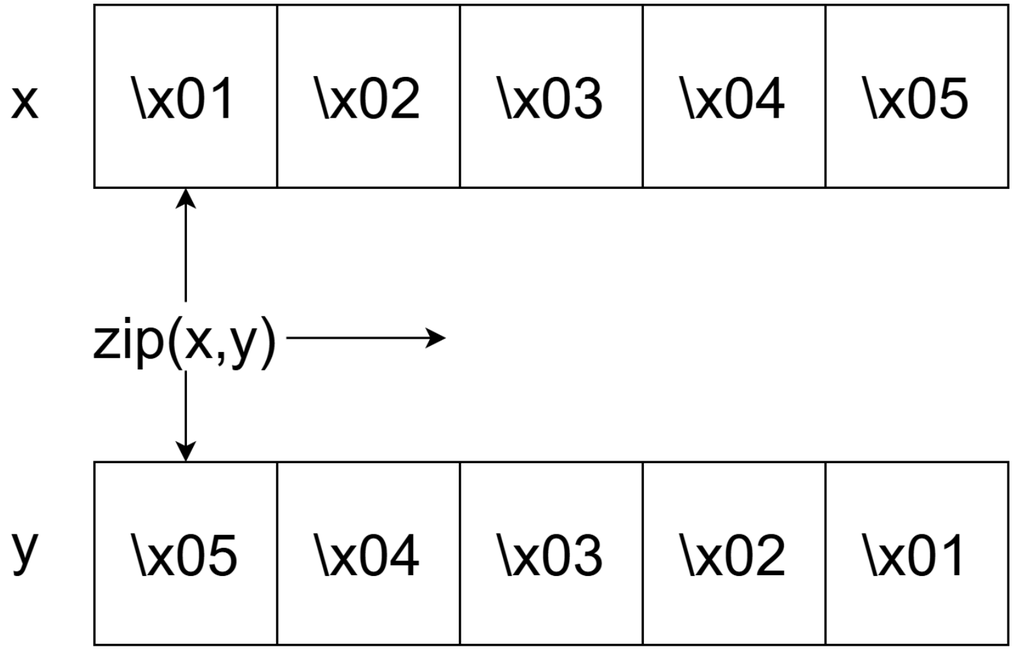
NOTE: b[0] extracts the byte as an integer.

XOR-ing Bytes Sequences

1: Using Python built-in methods

```
z = bytes([x ^ y for x, y in zip(x, y)])
```

- x and y are **byte objects**.
  - Think of them as rows of little boxes, each holding a byte.
- `zip(x,y)` goes through both "rows" together...
  - It takes the first box from x and from y, then the second, and so on.

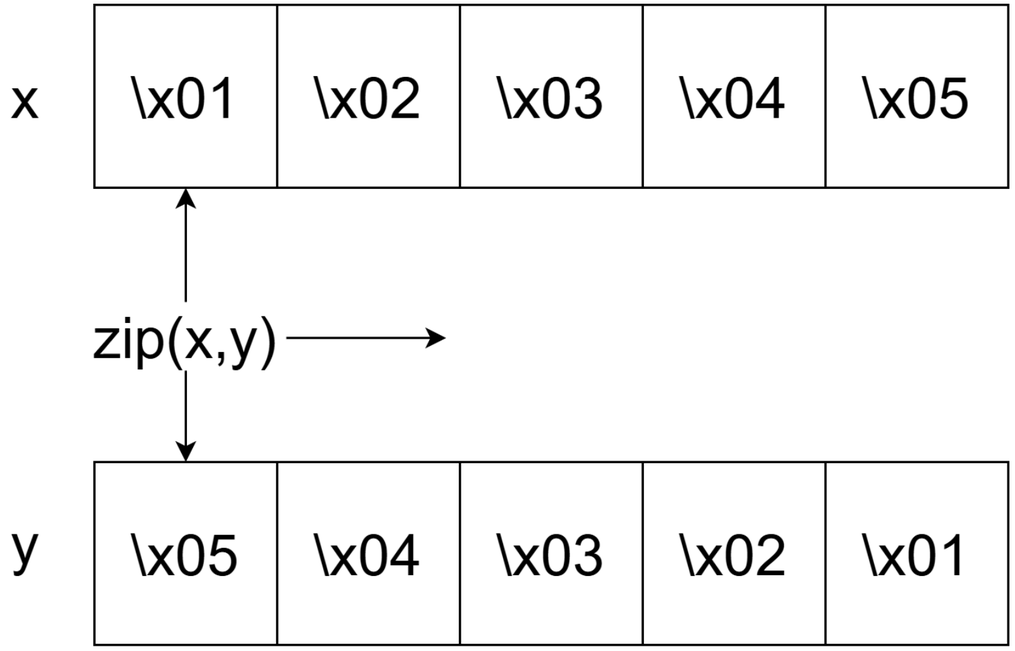


XOR-ing Bytes Sequences

1: Using Python built-in methods

```
z = bytes([x ^ y for x, y in zip(x, y)])
```

- `for x,y in zip(x,y)` -> for each pair of "boxes" in the same position...
- `x ^ y` performs the bitwise XOR.
- `[...]` collects all results into a list.
- `bytes([...])` turns the list into a byte object.



XOR-ing Bytes Sequences

2: Using PyCryptodome's strxor() function

1. Install the library: `pip install pycryptodome`
2. Import the function: `from Crypto.Util.strxor import strxor`
3. Use it to XOR together 2 bytes object **of the same length**

```
flag = strxor(key, encrypted_flag)
```

XOR-ing Bytes Sequences

3: Using pwntools' xor() function

1. Install the library: `pip install pwntools`
2. Import the function: `from pwn import xor`
3. Use it to XOR together 2 bytes object **of any length**
   - Byte objects of different lengths will produce results that contain garbage bytes (that you can ignore)!

```
flag = xor(key, encrypted_flag) #where len(key) > len(encrypted_flag)
```

Possible output:

```
WIT{th1s_is_th3_fl4g?!?}\x03\x10\xff
```

NOTE: garbage bytes at the end can be ignored.

RSA Essentials - Operators

> Below are the main arithmetic operators used in RSA computations.

Modulo (%)

```
a % n
# Example: 17 % 5 = 2
```

- Computes the **remainder** after dividing a by n.
- RSA encryption and decryption are **always done modulo n**.

Integer Division (//)

```
a // b
# Example: 17 // 5 = 3
```

- Division with the **remainder discarded**.
- Useful when working with prime factors and RSA key formulas.

RSA Essentials - Functions

> The following methods are useful when solving RSA challenges.

pow - **modular exponentiation**

```
pow(base, exponent, modulus)
```

Computes $(\text{base}^{\text{exponent}}) \bmod \text{modulus}$

Why it matters for RSA:

- Core operation in encryption $(c = m^e \bmod n)$
- Core operation in decryption $(m = c^d \bmod n)$
- **Handles very large integers efficiently**, which is essential for RSA

> NOTE: Do not use ** (Python's exponentiation operator) for large RSA numbers

```
m ** e % n   #Slow / not recommended
```

RSA Essentials - Functions

> The following methods are useful when solving RSA challenges.

`math.gcd` - **greatest common divisor**

```
import math

a = 12
b = 18
math.gcd(a, b) #returns 6
```

- Computes the **largest integer that divides both a and b**.
- Used to check if e and $\varphi(n)$ are coprime.

```
math.gcd(e, phi_n)  # should be 1 for RSA
```

PyCryptodome's helpers

Installation: `pip install pycryptodome`

Large Integers helpers

Allows easy **conversion between bytes and very large integers** used in crypto (RSA).

Bytes → Integer

```
from Crypto.Util.number import bytes_to_long

b = b'Hello'
n = bytes_to_long(b)
```

- Treats the bytes as one big integer.
- Preserves byte order (big-endian by default).

Integer → Bytes

```
from Crypto.Util.number import long_to_bytes

n = 310939249775
b = long_to_bytes(n)
```

- Converts a large integer back into bytes.
- Works for integers of any size.

NOTE: the above operations can also be performed using Python's built-in methods:

```
m_bytes = n.to_bytes((n.bit_length() + 7) // 8, byteorder='big')   #equivalent to long_to_bytes(n)
n = int.from_bytes(b, byteorder='big')                             #equivalent to bytest_to_long(b)
```

PyCryptodome's helpers

Modular Inverse helper

Computes the **modular inverse** of $a \bmod n$

```
from Crypto.Util.number import inverse
d = inverse(e, phi_n)  #same as pow(e, -1, phi_n)
```

- Essential for RSA private key computation $(d = e^{-}1 \bmod \varphi(n))$

Prime Numbers helpers

```
from Crypto.Util.number import getPrime, isPrime
p = getPrime(512)
isPrime(p)  # True
```

- `Crypto.Util.number.getPrime(bits)` generates a prime of bits length.
- `Crypto.Util.number.isPrime(n)` checks if a number is prime.
- Useful for RSA key generation or attacks on small primes.